

XPression 実務ハンズオン 手順書

DekoBokoTech / XPression実務ノート

本書は、同梱のサンプル素材を使って XPression の実務を手を動かして学ぶための手順書です。各ハンズオンのフォルダに対応する素材が入っています。

※サンプル素材はすべて自作のダミーです。実在のチーム・ロゴ・色は使用していません。

収録ハンズオン

No.	タイトル	レベル	目安	対応フォルダ
1	壊れないローワーサードを作る	中級	30分	01_LowerThird
2	スコアボードを作る	中級～上級	40分	02_Scoreboard
3	DataLinqで画像を差し替える	中級	25分	03_DataLinq_ImageSwap
4	簡易ゲームクロックを作る	中級	30分	04_GameClock
5	CSV／Excelでスコアや名前を流し込む	中級	35分	05_DataLinq_Practice
6	Staggerで一文字ずつ出す見出し	中級	25分	06_Stagger_Text
7	Transitionで出入りを設計する	中級	25分	07_Transition
8	スタッフロール（Roll）を作る	中級	30分	08_StaffRoll
9	Maskで見せる範囲を作る	中級	25分	09_Mask
10	Gradientで読みやすい色面を作る	初級	20分	10_Gradient
11	現在時刻・カウントダウンを表示する	中級	30分	11_Clock_Countdown
12	Render Viewで仮想モニターを作る	中級	30分	素材不要（機能だけで作成）
13	JSONで選手リストを回す	中級	40分	13_JSON_PlayerList
14	RossTalkで外部からTakeする	中級	30分	14_RossTalk

各ハンズオンの素材は、対応フォルダの中に入っています。手順を読みながら、その素材を使って進めてください。

1. 壊れないローワードを作る

ネイティブ機能とVisual Logicの分担

第4巻 第12章 | レベル：中級 | 目安 30分 | 素材フォルダ：01_LowerThird

ねらい

名前と肩書きの2段ローワード。短い名前でも長い名前でも背景が文字に合わせて気持ちよく収まり、いくら長くても安全エリアからはみ出さない上限を持つ。肩書きが空のときは行を出さず名前だけできれいに見せ、Online / Offline / Take で安定して出し隠しできる。

STEP 1 器を作る（背景とText Objectの親子）

この作例の器は「名前Text+その子の背景」と「肩書きText」。

- ① Object Library の Text をSceneにドラッグし、名前用Textを作って Object Manager で Name_TXT に改名
- ② 背景用オブジェクト（QuadやBackground）を追加し、Object Manager上で Name_TXT の子（First Child）になる位置へドラッグして親子化（この親子関係が後のAuto Scaleで効く）
- ③ 肩書き用Textを追加して Title_TXT に。仮の名前・肩書きを入れておく
- ④ 背景の色はMaterialで持たせる（色を「値」として持つと、後でチームカラー差し替えが楽）

確認：Object Managerで Name_TXT の子に背景がぶら下がり、名前だけで役割が分かる。

STEP 2 ネイティブで幅の土台を作る（ここが本命）

名前のText Object の Tabs & Options タブで3つを設定する。

- ① Auto Squeeze：Enabled にして Max Width に上限幅を入れ、Scaling は Width Only（横だけ詰める）。
 - ② Auto Scale：Enabled・Target = First Child・Mode = Width & Height にすると、親の詰まりに子の背景が追従する（背景幅をVisual Logicで計算する必要はない）。
 - ③ Character Limits：Soft Limit で警告（文字が赤くなる）、Hard Limit で文字数を頭打ち（最後の保険）。
- これで幅まわりは完成、Visual Logicは一切使わない。

確認：仮の長い名前を入れると横に詰まり、Max Widthを超えない。背景も一緒に追従する。Soft Limitを超えると文字が赤くなる。

ヒント：Max Width・Hard Limit

の具体値は安全エリアとフォント依存。実機で最長の名前を入れて決める。

STEP 3 Visual Logicの出番①：肩書きが空なら、行を出さない

「肩書きが空なら行ごと隠す」をVisual Logicで作る。

- ① Sceneを選択し、Visual Logic (Visual Logic Editor) を開く
- ② 左のObjectsから Title_TXT を展開し、Text (値) をキャンバスヘドラッグ
- ③ Function Blocks の Strings から String Length、Logic から Greater Than をドラッグ
- ④ 接続：Title_TXT.Text → String Length.In / String Length.Out → Greater Than.Base (しきい値0=「1文字以上あるか」)
- ⑤ Greater Than の出力を、肩書き行 (Title_TXTと背景をまとめたGroup) の Visible に接続
空なら0で非表示、入っていれば1で表示になる。名前を中央に寄せたいときは、同じ判定を名前Textの Position(Y) にもつなぐ。

確認：肩書きを空にすると肩書きの行が消え、入れると再表示される。

STEP 4 設計の判断：読めなくなる手前で切り替える (Visual Logicの出番②)

実機検証済みの構成をそのまま組む：

- ① Visual Logicで Name_TXT の Text → Strings の String Length に接続
- ② Math の Value を置き、しきい値の文字数を入力 (実機例 43)
- ③ String Length.Out → Logic の Greater Than.Base、Value → しきい値側に接続
- ④ Greater Than の出力 → Name_TXT の WordWrap.Enabled に接続 (しきい値超えで折り返しON)
- ⑤ Strings の String Value (実機例 1520) → WordWrap.Width に接続して折り返し幅を固定
折り返しそのものはネイティブの Word Wrap が担当し、Visual Logicは「いつ切り替えるか」の判断だけを持つ。

確認：しきい値を超える長い名前だけ2行に折り返され、読める状態を保つ。

ヒント：切り替えのしきい値は文字数で近似する。解像度・セーフティゾーン・フォント依存なので実機で決める。

STEP 5 出す・隠すを整える

- ① SequencerにSceneをドラッグしてTake Item化し、Template Linksで公開した名前・肩書きを差し替えて「短い名前」「長い名前」「限界超え」「肩書きなし」の4パターンのTake Itemを用意
- ② 送出はTake (テンキー+, RossTalkならTAKE)、消すのはOffline (テンキー-, SEQO)
- ③ 4パターンを順番に出して、どれも崩れず読めるかをリハで確認
- ④ 差し替えたら、プレビュー確認→再TAKEで送出に反映 (鉄則)

確認：全パターンで背景が追従し、安全エリアからはみ出さない。

本番前チェックリスト

- ☐ いちばん長く想定される名前で、Auto Squeezeが効いて安全エリアからはみ出さない
- ☐ 背景がAuto Scaleで名前に追従し、余白が破綻していない
- ☐ 極端に長い名前で、Hard Limitが最後の保険として効く

- ☐ 横詰めが効きすぎて読みにくくなる手前で切り替えが入る（収まる≠読める）
- ☐ 肩書き無し的时候、間延びせず名前だけきれいに収まる
- ☐ 送出中に名前を差し替えても、詰めと背景の追従が破綻しない

2. スコアボードを作る

Visual Logicが本領を出す場所

第4巻 第13章 | レベル：中級～上級 | 目安 40分 | 素材フォルダ：02_Scoreboard

ねらい

2チームのスコアボード。ホームとアウェイのチーム名・色・スコア・ピリオド表示。勝っている側がひと目で分かるリード強調。スコアが動いたときは変わった数字が軽く強調され、登場と退場はなめらかに。

STEP 1 器を作る（スコアボードのオブジェクト構成）

- ① 新しいSceneに、Text×4（HomeName_TXT／HomeScore_TXT／AwayName_TXT／AwayScore_TXT）、背景パネル、リードマーカー（HomeLead／AwayLead：グローや矢印）を左右対称に置く
- ② ピリオド用 Period_TXT、クロック用 Timer_TXT も配置
- ③ Object

Managerで、左右と役割が一目で分かる名前に統一する（あとの結線で迷わないための投資）

確認：Home/Awayの各部品が左右対称に置かれ、名前だけで役割が分かる。

STEP 2 スコアを表示用に整える

実機検証済みの構成（NumFormatタブ）：

- ① Visual Logic Editorでタブを追加し NumFormat と命名（機能ごとにタブを分けると後から読める）
- ② Strings の Format String を2つ置く
- ③ 接続：スコアの元値 → Format String.Param → SB_HomeScore_TXT.Text（Away側も同様）
- ④ 「07」のような桁揃えはFormat側の書式で決める。数値はそのまま出さず、表示の直前で整える

確認：スコアに1桁・3桁を入れても表示形式が意図通り。

ヒント：

桁数の想定（最大何点まで）とゼロ詰めの有無は競技・運用で決め、実機で桁あふれの挙動を確認する。

STEP 3 Visual Logicの出番①：リードを強調する

- ① Logic の Greater Than でホームとアウェイのスコアを比較
- ② 結果を HomeLead／AwayLead の Visible に接続（勝っている側だけ表示）
- ③ 同点分岐を最初から入れる：Equal To で同点を判定し、両マーカーを消す（または同点用の表示に切替）。放置すると「両方勝ち」に見える事故になる
- ④ 色の強弱も付けるなら、判定結果を Colors 系ブロック経由でMaterialへ

確認：ホーム優勢・アウェイ優勢・同点の3状態すべてで強調が正しく出る。

ヒント：比較・色切り替え・同点判定の具体的なFunction Blocks結線は、実機で組んだものを正とする。

STEP 4 Visual Logicの出番②：チームカラーを出し分ける

- ① チーム選択（WidgetのText Listや公開Text）を入力にして、Selectors の Input Selector で対応するチームカラーを選ぶ
- ② 選ばれた色を、背景・マーカーの Material（Colors.Diffuse）に接続
「どの色にするか」を決めるのがVisual Logic、「その色で表示する」のがネイティブ（Material）という分担。

確認：チーム選択を切り替えると、背景とマーカーの色が対応チームカラーに変わる。

STEP 5 動きをネイティブで付ける

実機検証済みの構成：

- ① 登場・退場は Scene Director にIn/Outの動きを作り、SequencerのTransition In/Out列と組み合わせる。順序はSequencerで管理
- ② スコア強調用に、Scene Director に ScoreMove（数字が軽く弾む短い動き）を作り、Event（Stop）で待機させる
- ③ Visual Logic に PointMove タブを作り、SB_HomeScore_TXT の Events.OnSetText → Scene Director（ScoreMove）の Actions.Play に接続（Away側も同様）
テキストが差し替わった瞬間に強調が自動再生される。値が動く瞬間に表示が破綻しないか必ず確認。

確認：スコアを変えた瞬間に強調が出て、表示が崩れない。

STEP 6 発展：スコアに比例したバーを作る（任意）

実機検証済みの構成（%BARタブ）：

- ① HomeScore と AwayScore → Math の Add（合計）
- ② HomeScore → Divide.Base、Add.Out → Divide.Divisor（ホーム側の比率 0～1）
- ③ 0対0対策：Add.Out → Equal To（==0）→ Input Selector
で「合計0なら0.5、それ以外は比率」を選ぶ（0除算の定石）
- ④ 選ばれた値 → Multiply（×バー最大幅 実機例1500）→ HomeBar_SHAPE.Width に接続
比率は0～1なので、最大幅で自然に頭打ちになる。

確認：大差のスコアでもバーが最大値で止まり、はみ出さない。0対0でも半々で表示される。

本番前チェックリスト

- ☐ ホームリード・アウェイリード・同点の3状態すべてで、強調が正しく出る
- ☐ 同点のとき、両方が勝っているように見えていない
- ☐ チームを差し替えたとき、色が正しく出し分けられる
- ☐ スコアが2桁から3桁になっても、桁あふれせず読める
- ☐ スコアが入れ替わった瞬間・同点になった瞬間に、表示が破綻しない
- ☐ 登場・退場がなめらかで、送出の順序が乱れない

3. DataLinqで画像を差し替える

Dynamic Material と Material Path（画像・ロゴの自動差し替え）

第5巻 第3章 | レベル：中級 | 目安 25分 | 素材フォルダ：03_DataLinq_ImageSwap

ねらい

チームロゴ・選手写真・スポンサー素材などの画像を、毎回手で貼り替えずに切り替えられるテンプレートを作ります。表示する画像を「ファイルの置き場所（パス）」で決めるようにし、さらに Widget やデータの値を変えるだけで別の画像へ入れ替わる状態を目指します。対戦カードや出演者が変わっても、画像を貼り替える手間がなくなります。

STEP 1 素材ファイルを整理する

まず、差し替えたい画像をあとで指定しやすい場所に整理します。プロジェクトフォルダの中に「Images」フォルダを作り、用途ごとにサブフォルダを分けて、次のようなファイルを用意します。

【準備するフォルダとファイルの例】

Images/Teams/

- TokyoTeamLogo.png
- OsakaTeamLogo.png
- NagoyaTeamLogo.png

Images/Players/

- Player_01_Photo.png
- Player_02_Photo.png

Images/Sponsors/

- Sponsor_Main.png

Images/Backgrounds/

- Arena_Background.png

大事なポイント：チームロゴは「チーム名+Logo.png」の形でファイル名を統一します（TokyoTeamLogo.png / OsakaTeamLogo.png …）。こうしておくと、⑤で Widget の値（TokyoTeam / OsakaTeam …）を変えるだけで、対応するファイルへ自動で切り替わります。

確認：Images/Teams などのフォルダに、命名規則どおりのファイルが並んでいる。

ヒント：フォルダ名・ファイル名は半角英数字で統一するとパス指定でトラブりにくいです。ファイル名の付け方（値+固定語）を先に決めておくと、後の自動切り替え（⑤）がそのままハマります。

STEP 2 画像を表示するオブジェクトを用意する

画像を映すためのオブジェクトを置きます。

① Object Library の Primitives > Quad

を Scene にドラッグ（チームロゴやスポンサー画像は四角い面の Quad が分かりやすい）

② Object Manager で「HomeLogo」など役割が分かる名前を付ける

すでにロゴ枠などがある場合はそれを使って OK。

確認：Quad が置かれ、HomeLogo など役割が分かる名前が付いている。

ヒント：「画像を貼る面」を用意するイメージ。まだ固定の画像が入っていても大丈夫です（次で動的に切り替えます）。

STEP 3 マテリアルを Dynamic Material に切り替える

画像を出したいオブジェクトを選択し、Object

Inspector（設定パネル）を開きます。「DataLinq」タブへ移動し、「Select Material

Source」で「Dynamic Material」を選びます。これで、このオブジェクトの画像が『固定』ではなく『後から指定して差し替えられる』状態になります。

確認：DataLinq タブの Select Material Source が Dynamic Material になっている。

ヒント：Dynamic Material＝『表示する画像を、パスやデータで動的に決められるマテリアル』。ここが画像差し替えの心臓部です。

STEP 4 Material Path で表示する画像を指定する

Dynamic Material Properties 中の「Material Path」に、表示したい画像ファイルの場所（パス）を入力します。プロジェクトフォルダからの相対パスで書くのが基本です。

記入例：

- ・チームロゴ → Images/Teams/TokyoTeamLogo.png
- ・選手写真 → Images/Players/Player_01_Photo.png
- ・スポンサー → Images/Sponsors/Sponsor_Main.png
- ・背景 → Images/Backgrounds/Arena_Background.png

ここで指定したファイルが、そのオブジェクトに表示されます。

【重要：編集画面（Layout）は市松模様でOK。送出側で表示される】

編集中の Layout ビューでは Dynamic Material はプレビューされず、青と黄色の市松模様（プレースホルダー）が表示されます。これは正常な挙動で、パスが間違っているわけではありません。実際の画像は送出（シーケンサー／Take＝オンエア）側で正しく表示されます。まずは Sequence で表示を確認してください。

【送出（Sequence）側でも画像が出ないときだけ、パスを見直す】

Sequence でも市松のまま／画像が出ない場合は、パスが解決できていません。次を確認します：

- ・パスの前後に "（ダブルクォート）が付いていないか。「パスのコピー」で貼り付けると "C:/…/TokyoTeamLogo.png" のように付くことがあり、残っていると解決できません。必ず削除。
- ・区切りが¥ではなく / になっているか。
- ・フォルダ名・ファイル名に日本語や全角が混じっていないか（半角英数字が安全）。
- ・指定したファイルが実際にその場所にあるか、名前・拡張子（大文字小文字含む）が一致しているか。
- ・相対パスの場合、プロジェクトを保存し、Images フォルダが保存先プロジェクトの下にあるか。（編集画面で位置やサイズを合わせたいときは、作業中だけ Static Material に同じ画像を割り当て、送出前に Dynamic Material へ戻すと確認しやすいです。）

【パスをコピーする方法（Windows）】

- ・エクスプローラーで対象ファイルを Shift+右クリック → 「パスのコピー」を選ぶ。
- ・フォルダのパスだけ欲しいときは、アドレスバーをクリックしてコピー。

コピーされるのは絶対パスです。Material Path を相対にしたいときは、プロジェクトフォルダより下の部分（例：Images/Teams/…）だけを残して貼り付けます。区切りは / でOKです。

- ・重要：「パスのコピー」で貼り付けると、パスの前後に "（ダブルクォート）が付くことがあります（例："C:/…/TokyoTeamLogo.png"）。この " は必ず削除してください。残っているとパスを解決できず、市松模様のままになります。

確認：Material

Pathに直書きしたパスでロゴが表示される（市松模様のままなら「"」や誤字を疑う）。

ヒント：絶対パス（D:/… など）を使う場合は、本番機でも同じパスで解決できるか必ず確認を。相対パスの方が別PCへ移しても崩れにくく安全です。

STEP 5 Widget (Text List) の値でファイル名を切り替える (具体的な書き方)

ここが自動切り替えの肝です。まず切り替え用の Widget を作ります。

Display メニュー → Widgets パネルの『New Widget』 → 『Text List』を選びます (=テキストを切り替えられるウィジェット。時刻なら Clock Timer、数値なら Counter)。

できた『TextList1』は、分かりやすい名前 (例: TeamNameForFile) に変更しておきます。

次に Widget Properties (プロパティ) を開き、『Add』で切り替えたい値を登録します。

例: TokyoTeam / OsakaTeam / NagoyaTeam …

※『Allow manual entry of text』にチェックすると、一覧に無い値もその場で手入力できます。

そして Material Path のファイル名部分に、この Widget を差し込むマクロ @W:ウィジェット名@ を書きます。

記入例:

Images/Teams/@W:TeamNameForFile@Logo.png

【手入力しない方法: 右クリックで挿入】

マクロは手で打たなくてもOKです。Material Path の入力欄を右クリック → 『Insert

Lookup』 → 『Widgets』 → 作成した

Widget名 (例: TeamNameForFile) を選ぶと、@W:TeamNameForFile@ が自動で挿入されます。

同じ右クリックメニューから『Published Text Objects』『All Text Objects』を選べば、テキストオブジェクトの値も同じように差し込めます。打ち間違い (スペルミス) を防げるので、こちらが確実です。

@W:TeamNameForFile@ の部分が、いま選ばれている値に置き換わります。

- 選択が TokyoTeam のとき → Images/Teams/TokyoTeamLogo.png
- OsakaTeam に切り替えると → Images/Teams/OsakaTeamLogo.png

Widgets パネルのドロップダウンや Prev / Next

で値を切り替えると、表示ロゴが自動で入れ替わります。

確認: Widgetの値を切り替えると、ロゴが自動で入れ替わる。

ヒント: 値の種類で使うWidgetが違います: 文字 (チーム名など) =Text

List、数値カウント=Counter、時刻=Clock

Timer。コツはファイル名を『値+固定語』で統一すること (TokyoTeamLogo.png / OsakaTeamLogo.png…)。外部データで切り替えるなら @W:名前@ の代わりに %DataLinqキー名% を使います。

STEP 6 本番前に確認する

想定するすべての画像で、パスが正しく解決できて画像が表示されるかを確認します。特に次の3点をチェック：

- ・ 相対パスが別PC（本番機）でも通るか（保存場所や共有ドライブの割り当て違いに注意）
- ・ Widget／データの値を変えたら、本当に画像が切り替わるか
- ・ 指定した画像が見つからない（空・欠番）ときに大きく崩れないか

リハーサルで実際に何パターンか切り替えてみると安心です。

ヒント：「本番機で画像が出ない」の多くはパスの解決ミス。素材はプロジェクトに同梱し、相対パスで指定しておくことで事故が減ります。素材の置き場所が毎回変わる現場では file path 指定は慎重に。

本番前チェックリスト

- ☐ 相対パスで、別のPC（本番機）でも画像が表示される
- ☐ Widget (@W:名前@) またはDataLinqキー (%キー名%) の値を変えるだけで画像が切り替わる
- ☐ ファイル名を『値＋固定語』で統一してある (TokyoTeamLogo.png など)
- ☐ 絶対パスを使う場合、本番機でも同じパスで解決できることを確認した
- ☐ 指定した画像が無いとき（空・欠番）に表示が大きく崩れない
- ☐ 対戦カード・出演者の差し替えを、手作業なしで回せる

4. 簡易ゲームクロックを作る

Clock Timer Widgetでカウントダウン表示

第4巻 第14章 | レベル：中級 | 目安 30分 | 素材フォルダ：04_GameClock

ねらい

12:00から00:00へ向かってカウントダウンする簡易ゲームクロック。Text Objectは表示先で、時間の値はClock Timer Widgetが持つ。Start / Stop / ResetをManualで確実に操作でき、Online・再Takeでも意図しないResetや再Startが起きないことを確認する。

STEP 1 作り方を3つに切り分ける

時計・タイマー表示は目的で道具が変わる。

- ① シンプルなカウントダウン→Clock Timer Widget（今回はこれ）。
- ② 残り時間で色を変えるなどの演出連動→Visual Logic。
- ③ 外部スコアボード等の公式時計を表示→DataLinq。ここでは最も分かりやすいClock Timer Widgetで作る。Visual Logicを無理に最初から使わない。

確認：「今回はClock Timer Widgetで作る」と方針を言える。

STEP 2 テスト用Sceneを作る

- ① Scene Managerで新規Sceneを作成し、GameClock_Test など分かる名前を付ける
- ② この段階では背景・装飾・ロゴ・スコアは入れない。まず確認したいのは見た目ではなく「時計が正しく表示され、Start / Stop / Resetできるか」

確認：GameClock_Test という空のSceneができている。

STEP 3 表示先のText Objectを置く

- ① Object Library の Text
をSceneにドラッグして1つ置き、仮で「12:00」と入れる（位置・サイズ・Font・Materialの確認用）
- ② 名前は Clock_Text のように役割が分かる名前にする
重要：この仮文字を打っただけでは動くタイマーにはならない。Text Objectは時間を作る部品ではなく、時間の値を表示する表示先。

確認：Clock_Text に仮の「12:00」が表示されている（まだ動かなくてよい）。

STEP 4 Clock Timer Widgetを作る

メニューの Display > Widgets を開き、New Widget から Clock Timer を作る。名前は GameClock_Timer のように用途が分かる名前に（後で Text Object 側から選ぶときに効く）。Widget s画面は『Timerを作る場所』であり、同時に『Timerの状態を見て操作する場所』でもある。

確認：Widgetsパネルに GameClock_Timer が並んでいる。

ヒント：Widgetが増えるとどのTimerをどのTextにつないだか分からなくなる。最初から用途の分かる名前を付けておく。

STEP 5 ModeをTimerにする

作ったClock Timer WidgetのPropertiesで Mode を Timer にする。Clock＝『今の時刻』を扱う考え方、Timer＝『任意の開始値から進める／戻す』考え方。12:00から00:00へ減らしたいのでTimerが中心。

確認：Mode が Timer になっている。

STEP 6 Start At / Stop At / Direction を決める

12分→0分のカウントダウンなら、Start At=12:00、Stop At=00:00、Direction=Down。暗記すべきは設定名ではなく『どこから始まる／どこで止まる／上がるか下がるか』の3つ。入力形式や項目名はバージョン・画面で異なるので実機で確認する。

確認：Start At=12:00・Stop At=00:00・Direction=Down になっている。

ヒント：この3つを間違えると、表示形式が合ってもタイマーとして使いにくくなる。

STEP 7 Formatを決める（まずはNN:SS）

FormatはTimerの値をどんな文字列で出すかの設定。NN:SS / HH:NN:SS / NN:SS.ZZZ などが扱える。簡易ゲームクロックは NN:SS（例 12:00 / 05:30 / 00:10）が分かりやすい。小数表示（.ZZZ）は桁が増え、Fontによっては数字の幅が揃わず揺れて見えるので、最初はNN:SSが安全。

確認：表示が NN:SS 形式になっている。

STEP 8 Start / Stop / Reset を Manual にする

最初の作例では Start / Stop / Reset の Method を Manual にする（Operatorが明示的に操作）。When Onlineにすると出した瞬間に動き出し、原因の切り分けがしにくい。注意：『Manualに設定する』ことと『実際にStartする』ことは別。StartはあとでWidget側の操作で実行する。Ctrl+1～9などショートカット割り当てはKeyboard Mappingと競合しないか実機で確認。

確認：Start / Stop / Reset の Method がすべて Manual になっている。

STEP 9 Text ObjectのData SourceにWidgetをつなぐ

Clock_Textを選び、Text Objectの Data Source タブを開く。
Data SourceとしてWidgetを選び、一覧から GameClock_Timer
を選ぶ。これでClock_Textにタイマーの値が表示される。
直接「12:00」と打ちっぱなしにしない（それは固定表示）。

表示されないときの確認順：

- 対象のTextを選択しているか
- Data SourceでWidgetを選んだか
- 正しいWidget（GameClock_Timer）を指定したか
- Formatは意図通りか
- Font／Material／位置に問題はないか

確認：Clock_Text にWidgetの値（12:00）が表示されている。

STEP 10 Main Viewportで表示を確認する

接続できたらMain Viewportで値が出ているか確認。ここで見るのはデザインではなく『Widgetの値が表示されているか／Formatが意図通りか／文字が範囲に収まるか』。動きが見えないときはViewport上部の Show or Hide Continuous Animations and Other Effects などの表示更新状態も確認。表示されている＝Startしている、ではない点に注意。

確認：Main Viewportで値とFormatが意図通りに見えている。

STEP 11 Widget側のManual操作でStart→Stop→Reset

Widget側の操作でStartし、12:00 → 11:59 → 11:58 と減れば表示接続は成功。

動かないときは、Text Objectのデザインより先に次を確認：

- WidgetがStartしているか
- MethodがManualになっているか
- Start操作を実行したか
- Start At・Directionが意図通りか

次にStopで保持、Resetで開始値へ戻るかを確認する。

Resetは強力（本番中の誤操作で意図しない値に戻る）。「いつ／誰がResetしてよいか」「自動Reset（Reset on timer 等）が有効になっていないか」まで確認する。

確認：Startで減り始め、Stopで止まり、Resetで12:00に戻る。

STEP 12 OnlineにしてTake/Offline/再Takeを確認する

Layoutで確認できたらSceneをOnlineにして、送出時の挙動を見る。Main

Viewportで動いても、送出時に同じとは限らない。

確認すること：

- Onlineにしたとき時計は止まっているか
- Manual Startまで動かないか
- Offlineで止まる・保持・Resetのどれになるか
- 再Take時に意図しないReset・再Startが起きないか

再Take時の値保持/ResetはScene属性やSequencer・Widgetの設定に依存するので必ず実機で確認し、Operatorが説明できる状態にしておく。

本番前チェックリスト

- ☐ 3つの作り方のうち、簡易＝Clock Timer Widgetを選んでいる
- ☐ Text Objectは表示先、時間の値はClock Timer Widgetが持つ、と分けて作っている
- ☐ Start At / Stop At / Direction が意図通り（どこから・どこで止まる・増減）
- ☐ Manual設定と実際のStart操作を混同していない（表示≠Start）
- ☐ Reset のタイミングと権限、自動Resetの有無を確認した
- ☐ Online・Offline・再Takeで意図しないReset/再Startが起きないことを確認した

5. CSV／Excelでスコアや名前を流し込む

DataLinqとWidgetsで手入力を減らす

第5巻 第3・4章 | レベル：中級 | 目安 35分 | 素材フォルダ：05_DataLinq_Practice

ねらい

外部の表（CSV/Excel）から選手名・スコア・チーム情報をText Objectへ流し込み、表を書き換えるだけで表示が変わる状態を作る。データソースは『誰が編集し、どう壊れるか』で選び、失敗時の見え方まで想定する。

STEP 1 DataLinqの共通構造を押さえる

DataLinqは『外部データを取り込み→キー（列や項目）で参照→Text ObjectやMaterialに表示』という共通構造。形式（CSV/Excel/JSON）が違ってても骨格は同じ。まず『何の値を、どのキーで、どこに出すか』を決める。

確認：「何の値を・どのキーで・どこに出すか」を言葉にできる。

STEP 2 まずCSVで動かす

- ① XPression DataLinq Server で Add DataLinq Source → Text DataLinq Source を追加（CSV＝区切り文字付きテキストはこれ。一覧に実在確認済み）
- ② File Pathに teams.csv を指定し、Encoding（UTF-8）・区切り文字・ヘッダーの扱いを確認
- ③ XPression側：Text Objectの Data Source タブ → DataLinq を選択 → Set → Sourceを選び Browse → 表から表示したいセルを選ぶ（Column＝列番号／Row＝行番号）
まず1件表示して、文字化け・列ずれがないか確認する。

確認：CSVの1件目（Row＝2）のチーム名がText Objectに表示された。

ヒント： CSVは半角英数字の列名・UTF-8がトラブりにくい。1行目（ヘッダー）がキーになる。

STEP 3 行を切り替えて表示を変える

読み込んだ表の「どの行を表示するか」を切り替えると、名前やスコアが差し替わる。

【実機確認済みの行番号ルール】CSVでは Set DataLinq Properties の Row=行番号、Column=列番号。ただし1行目はヘッダー行（TeamName等の見出し）としてカウントされるため、データ1件目は Row=2 になる（例：Row=2でTokyoTeam、Row=3でOsakaTeam）。JSONの players<2>（ヘッダー概念なし）とは数え方が違う点に注意。ローワーサードなら1件ずつ、スコアボードならHome/Awayの2件を行違いで割り当てる。表（CSV）側を書き換えて再読み込みし、表示が追従するか確認する。

確認：Rowを2→3に変えると表示が次のチーム（人）に切り替わる。CSVを書き換えて再読み込みすると追従する。

ヒント： 行の切り替えを運用で楽にしたいときは、JSONハンズオンと同じDataLinq Key方式（Row欄に%キー名%）が使える。

STEP 4 編集しやすさが要るならExcel、構造化ならJSON

現場担当者が頻繁に手編集するならExcelが選ばれやすい（ただし、どのシート・範囲・保存形式を読むかの読み取り方式を実機で確認）。項目が入れ子・外部システム連携ならJSON。断定より『現場で確認すべき観点』で選ぶ。

確認：自分の現場ならどれを選ぶか、理由付きで説明できる。

STEP 5 本番挙動（更新・失敗時）を決める

Live Update（自動更新）は便利だが全項目で使うものではない。Poll every（取得間隔）は短ければよいわけではない。Return Empty on Failure（失敗時に空を返す）で、データが取れないときの見え方が決まる。取れないとき前の値が残るのか空になるのかを想定して選ぶ。

確認：データを読めなくしたとき（ファイル名変更など）の表示が想定どおり（空／前回値）。

ヒント： 『空データ時にどう見えるか』を必ず一度作って確認。Prepend/Appendを使う場合は空データ時の挙動に注意。

STEP 6 Widgetsで切り替えも足す（任意）

手入力を減らす仕上げとしてWidgetsが使える：

① Display > Widgets を開き、New Widget から作る：Text

List（選択肢の切替）／Counter（スコアの増減）／Clock Timer（時刻・タイマー）

② Widgetは作っただけでは表示に出ない。Text ObjectのData Source タブ → Widget
を選んで割り当てて初めて効く

DataLinq（外部データ）とWidget（現場操作の値）は分けて考える。

確認：Widgetの値がText Objectに表示され、切り替えで変わる。

STEP 7 本番前に確認する

データソースは『誰が編集し、どう壊れるか』で選ぶ。確認：想定する全データで正しく表示されるか／文字数が多い行でも崩れないか／データが取れない・空・欠番のときに大きく破綻しないか／編集ルール（列を消さない等）を担当者が守れるか。

本番前チェックリスト

- ☐ 『何の値を・どのキーで・どこに出すか』を先に決めている
- ☐ CSVで1件表示→行切り替えで差し替わることを確認した
- ☐ 形式（CSV/Excel/JSON）を『編集頻度・構造・連携』で選んでいる
- ☐ Live Update / Poll every / Return Empty on Failure の挙動を決めた
- ☐ 空データ・欠番時の見え方を確認した
- ☐ Widgetを使う場合、Data Sourceへ割り当てて表示に反映している

6. Staggerで一文字ずつ出す見出し

現場で読めるテキスト演出

第3巻 第2章 | レベル：中級 | 目安 25分 | 素材フォルダ：06_Stagger_Text

ねらい

文字が少しずつずれて登場する見出し。設定を作る→Scene

Director上の対象へ配置→再生して確認、の3ステップで、Timing Offsetsで『文字ごと／単語ごと／行ごと』のずれを設計する。差し替わっても成立する演出にする。

STEP 1 3ステップの流れを押さえる

Stagger Animationは①アニメーションの設定を作る ②その設定をScene

Director上の対象の行へ配置する ③再生して見え方を確認する、の3つ。

設定は右ドックの Stagger Animations パネル（New…／Edit…）で作る（実機確認済み）。

確認：「①設定を作る→②配置する→③再生する」の3ステップを言える。

STEP 2 対象と名前を決める

動かすText ObjectをSceneに置く。オブジェクトは作った時点でScene Directorに行（トラック）が自動で用意されるので、手動で追加する必要はない（実機確認済み）。手動でやるのは、作ったStagger Animationを対象Textの行に配置するほう。

Stagger AnimationのNameとDescriptionは現場運用のために使う（後から見て何の動きが分かるように）。リアルタイムでは、きれいさだけでなく「あとから見ても分かる状態」が大事。

確認：Scene Directorに対象Textの行があり、Stagger AnimationのName/Descriptionで用途が分かる。

STEP 3 Trackで『何を動かすか』を決める

① Stagger Animations パネルで New… → できた StaggerAnimation1 を選んで Edit…

② Edit Stagger Animation の Tracks で、動かすプロパティを追加（実機例 Position.X。登場演出なら Alpha+Position が定番）

動かす対象のプロパティだけTrackを持たせる。

確認：Tracksに意図したプロパティ（例 Position.X）だけが並んでいる。

STEP 4 Keyframeで動きの形を作る

- ① 同じ Edit Stagger Animation の Track Animation で、変化のカーブを作る（例：Position.X を画面外から0へ）
- ② Total Duration の Frames が全体尺（実機例 150）。1文字ぶんの変化の長さ（Clip Length）と全体尺は分けて考える
尺を変えたら Recalculate Keyframe Positions で位置ずれがないか確認。

確認：再生すると1つの塊として動く（まだ、ずれ無しでOK）。

ヒント：Track Controlsは作業中の事故（誤って別Trackを動かす等）を減らすために使う。

STEP 5 Timing Offsetsでずれを作る（ここが中心）

- ① 同ダイアログの Timing Offsets に、ずらすフレーム数を入れる（実機例：Character 3.00／Word 6.00／Line 15.00／Paragraph 30.00）
- ② Pivots の Mode（実機例 Baseline Center）は、ScaleやRotationを使うときの中心
- ③ できたStagger Animationを、Scene Director の対象Textの行ヘドラッグして配置する
日本語見出しなら Character か Word のずれが読みやすいことが多い。

確認：Character／Word単位で文字が順番に出てくる。

STEP 6 再生して読めるか確認する

- ① Scene Director の再生（または再生ヘッ드의ドラッグ）でプレビューを確認
- ② 「演出として気持ちよいか」だけでなく「読めるか」を見る。速すぎると文字が追えない
- ③ Template Linksで文字を差し替え、長い名前・短い名前でも破綻しないか確認

確認：再生途中でも文字が読め、差し替えても崩れない。

本番前チェックリスト

- ☐ 設定を作る→対象へ配置→再生確認、の3ステップで進めた
- ☐ Name / Description を後から分かるように付けた
- ☐ 動かすTrack（Alpha/Position等）を必要な分だけ持たせた
- ☐ Clip LengthとTotal Durationを分けて考え、尺変更後に再計算で確認した
- ☐ Timing Offsetsのずれ単位（Character/Word/Line/Paragraph）を選んだ
- ☐ 文字数が変わっても読める速度・崩れないことを確認した

7. Transitionで出入りを設計する

正しく消えて、正しく更新できるSceneにする

第3巻 第8章 | レベル：中級 | 目安 25分 | 素材フォルダ：07_Transition

ねらい

Sceneの登場（In）だけでなく退場（Out）と更新（Back-to-Back）まで設計し、本番で残り方が読みにくくならないようにする。Layerと重なり順、外部制御での消え方まで想定する。

STEP 1 Transitionが設計するものを理解する

Transitionは単なるエフェクトではなく、Sceneの出入りを破綻させないための設計項目。In（出る）だけでなく Out（消える）・更新（連続で内容が変わる）まで含む。

確認：In／Out／更新の3つを設計対象として言える。

STEP 2 In / Out をまず作る

設定場所は2つ（実機確認済み）：

① Sequencer の Transition In / Out 列 — Take

ItemごとにInとOutのエフェクト（Cut／Dissolve／Push など）を設定する。既定は Cut / Cut

② 凝った出入りは Scene Director にIn/Outの動きとして作る（条件で使い分けるならObject InspectorのTransition Logicタブ）

まずはInとOutの両方を必ず設定する（Outを設計しないと本番で残り方が読めない）。

確認：InとOutの両方が設定され、出して消すまでが成立している。

STEP 3 Durationは現場テンポで決める

① Sequencer の Transition In/Out 設定で長さを調整する

② 判断基準は見た目の好みではなく現場テンポ：常駐テロップ=自然にOut／緊急表示=即時（Cut）で消せる経路／演出Scene=Outまで含めて見せ方を作る／スコア更新系=次の情報を邪魔しない長さ

確認：用途（常駐／緊急／演出）に合ったDurationになっている。

STEP 4 更新（Back-to-Back）を設計する

同じSceneで内容が連続更新される場合（スコアや情報更新）の見え方を決める：

① 同じTake

Itemの値を差し替えて再TAKEし、Back-to-Back（連続送出）でどう見えるかをリハで確認

② 「毎回In/Outを打つ」か「中身だけ差し替える」かを決めて統一する

確認：連続更新したときの見え方が意図どおり。

STEP 5 Scene DirectorとTransition Logicを分ける

役割を分けて考える：Scene Director＝動きの中身を作る場所／Transition Logic（Object Inspector - Sceneのタブ）＝条件に応じてどのScene Directorを使うかを決める場所。

単純なIn/Outで足りるならSequencer列だけで済ませ、必要な複雑さに留める（作り込みすぎると読めないデータになる）。

確認：どのScene Directorがいつ使われるかを説明できる。

ヒント：

本番前修正・別担当・複製・Layer変更・外部制御変更が起きる前提で、必要以上に高度にしない。

STEP 6 LayerとFramebuffer・外部制御まで見る

動きだけでなく重なり順（Layer / Framebuffer）まで見る。別Layerの状態に反応させるならScene Triggersを検討。外部制御（GPI / RossTalk / PBus等）で消す場合は、Transitionの想定と実際の消え方が一致しているか確認。

確認：重なり順と、外部制御で消したときの動きが想定どおり。

STEP 7 本番前の確認順序

In だけを見ず、更新・Out・Layer・外部制御まで含めて確認する。確認：出たあと正しく消えるか／連続更新が破綻しないか／重なり順が意図通りか／外部制御で消したとき想定通りか。

本番前チェックリスト

- ☐ In と Out の両方を作った（Outを設計している）
- ☐ Durationを現場テンポ・用途で決めた
- ☐ 連続更新があるならBack-to-Backを設計した
- ☐ Scene DirectorとTransition Logicの役割を分けている
- ☐ Layer/Framebufferの重なり順を確認した
- ☐ 外部制御での消え方が想定と一致することを確認した

8. スタッフロール（Roll）を作る

Scene GroupでRoll/Crawlを設計する

第3巻 第9・10章 | レベル：中級 | 目安 30分 | 素材フォルダ：08_StaffRoll

ねらい

スタッフ名・役職が縦に流れるエンドロール。『どう動かすか』より『どう直すか・どう運用するか』を先に決め、Global Marginsで安全に見える範囲を保ち、Loopの止め方まで設計する。

STEP 1 用途を先に決める

Scene Groupを作る前に『何を、どう流すか』を決める。スタッフロールならスタッフ名・役職を表示するSceneをまとめる。スポンサーRollならロゴ入りSceneをまとめる。地味だがここが一番大事。作るときは『どう動かすか』より『どう直すか（本番前の修正・別担当・再利用）』を先に考える。

確認：何を・どの順で流すかが決まっている。

STEP 2 Scene GroupとSequenceのどちらで作るか選ぶ

Scene Group＝デザインのまとまりを優先、テンプレート構造に近い場所で管理。Sequence側（Take Item Group）＝内容変更が多い運用向き、送出しの並びで管理。今回はScene Groupで作る。どちらでも『あとから直す人が理解できる構造』にする。

確認：Scene Groupで作る理由を言える。

STEP 3 Scene Groupを作り、中に「ページ」を積む

ロールの実体は「Scene Groupの中に積んだ子Scene」で作る（実機確認済みの構造）。

- ① Scene ManagerでScene Groupを新規作成し、EndRoll_Staffのように後から見えて分かる名前を付ける
- ② そのグループの中に子Sceneを追加していく。子Scene1枚＝ロールの1ページ。Scene Managerで上から並んだ順番どおりに流れていく
（例：1枚目＝タイトルページ、2枚目＝スタッフ名一覧、3枚目＝スポンサー/締めページ）
- ③ 各ページの中身は普通のSceneと同じ作り方（Textを置いて名前・役職を並べる）。1ページの高さ＝Sceneの高さなので、長いスタッフ一覧はページを分けて積む
- ④ 設定はグループ側に集約されている：Scene Groupを選択 → Object Inspectorの「Scene Group」タブが、Roll化・速度・余白・ループすべての入口

確認：Scene

ManagerでEndRoll_Staffの下に子Sceneがぶら下がり、順番が意図どおりに並んでいる。

ヒント：グループ名が曖昧だと、修正・再利用の段階で誰も触れなくなる。ページの並び替えはScene Manager上の順番で決まる。

STEP 4 EffectでRoll（縦）かCrawl（横）を選ぶ

Scene Groupを選択し、Object Inspectorの Scene Group タブ →
Group欄で設定する（実機確認済み）：

- Effect：Roll（縦に流す）／Crawl（横に流す）。スタッフロールはRoll
- Direction：Bottom To Top（下から上へ＝視線の流れ）

確認：Effect=Roll・Direction=Bottom To Top になっている。

STEP 5 Duration（速度・時間感）を決める

手順3で開いた Scene Group タブの Duration

欄で決める（実機確認済み：Speed／Seconds／Framesのラジオ選択）：

- ① まず Speed（実機例 2.000）で大きな見え方を作る
 - ② 秒数で管理したいなら Seconds、最後の詰めは Frames
- 速すぎると読めず、遅すぎると間延びする。読みやすさ優先で。

確認：最後まで読める速度になっている。

ヒント：最初はSpeed/Secondsでざっくり、そのあとFramesで微調整。

STEP 6 Global Marginsで安全な範囲を保つ

同じ Scene Group タブの Global Margins

欄（Top／Bottom／Left／Right）で、画面端に対して安全に見える範囲を作る。
テロップが端で切れたり、はみ出したりしないための運用上の設定。

確認：画面端で文字が切れず、安全な範囲に収まっている。

STEP 7 開始・終了の空白とLoopを設計する

同じ Scene Group タブで設定する（実機確認済みの欄構成）：

- ① Header / Footer 欄：Blank Page on Start / Blank Page on End
にチェック（いきなり文字が入る／唐突に切れるのを防ぐ）
- ② Loop 欄：Enable Looping と Number of shows per
scene。Loopは便利だが「本番でどう止めるか」を決めてから使う
- ③ Start / Stop 欄：Ease In / Out（Frames）は気持ちよさより読みやすさで判断

確認：開始・終了が唐突でなく、Loopの止め方が決まっている。

STEP 8 本番前に確認する

確認：最後まで読める速度か／端で切れず安全範囲に収まるか／開始と終了が唐突でないか／Loopする場合に意図通り止められるか／Per Scene

Lightingなどで見え方が変わらないか。差し替え（名前追加）しても崩れないかも見る。

本番前チェックリスト

- ☐ 『どう動かすか』より『どう直すか・運用するか』を先に決めた
- ☐ Scene GroupとSequenceのどちらで作るか選び、理由が説明できる
- ☐ 後から分かる名前を付けた
- ☐ 速度は読みやすさ優先（Speed/Seconds→Framesで調整）
- ☐ Global Marginsで端で切れない
- ☐ Loopの止め方・開始終了の空白まで設計した

9. Maskで見せる範囲を作る

隠す・抜く・見せる範囲を設計する

第2巻 第14章 | レベル：中級 | 目安 25分 | 素材フォルダ：09_Mask

ねらい

四角い表示範囲（ワイプ／切り抜き）を作り、その範囲を動かせるようにする。『消す』ではなく『見せる範囲を決める』考え方で、本番で調整が要る値だけをPublishして運用を壊さない。

STEP 1 Maskの考え方を押さえる

Maskは『消す』のではなく『見せる範囲を決める』。何を見せて何を隠すかを先に決めてから作ると迷わない。使う前に『どの範囲を、いつ、どう見せたいか』を確認する。

確認：見せる範囲と隠す範囲を言葉にできる。

STEP 2 Layer Objectで対象をまとめる

- ① Object Library の Layers > Layer Object をSceneに追加
- ② マスクを効かせたいObjectたちを、Object Manager上でLayer Objectの子になる位置へドラッグ
LayerはMaskの効き先を区切る単位（公式ヘルプ確認済み：あるLayerの中だけをマスクし、他のオブジェクトやLayerに影響させない）。

確認：対象ObjectがLayer Objectの子にまとまっている。

STEP 3 Box Maskで四角い表示範囲を作る

- ① Box Mask オブジェクトをSceneに追加し、Layer内に配置する（追加はObject Libraryから。環境により表記を確認）
- ② 四角の内側＝見える範囲。Transform（Position／Scale）で範囲の位置と大きさを調整
画像の形で抜きたい場合は、Mask Material＋アルファ素材の考え方を使う。

確認：Box Maskの内側だけが見えている。

ヒント： まずBox Maskで四角の範囲を確実に作ってから、必要ならアルファ素材の抜きへ進む。

STEP 4 Scene DirectorでMaskを動かす

- ① Scene Director の Box Mask
の行に、Position（またはScale）のアニメーションを作る（例：左から右へ広がるワイプイン）
- ② 再生して「見えてくる／隠れる」動きになるか確認

確認：再生すると範囲が動き、ワイプとして見える。

STEP 5 調整が要る値だけPublishする

- ① 本番で調整する値だけを、対象オブジェクトの Template Links タブでチェックして公開する
 - ② 公開した値はSequencerのTake Inspectorに並ぶ。運用者が意味を理解できる名前にしておく
- 判断基準：本番中に本当に調整するか／制作用の値を残していないか／触ってはいけない値まで公開していないか／初期値に戻せるか

確認：Publishした値だけがTake Inspectorに出て、意味が分かる名前になっている。

STEP 6 本番前チェックと切り分け

確認：見せたい範囲だけが見え、隠したいものが隠れているか／範囲を動かしたとき破綻しないか／Publishした値が運用者に分かるか。うまくいかないときは、Layer構成・Mask種別・重なり順の順で切り分ける。

本番前チェックリスト

- ☐ 『消す』ではなく『見せる範囲を決める』で設計している
- ☐ 対象をLayer Objectでまとめた
- ☐ Box Maskで四角い表示範囲を作れた
- ☐ Scene Directorで範囲を動かせる
- ☐ 本番で調整する値だけをPublishし、触らせたくない値は隠した
- ☐ 初期値へ戻せる／項目名が運用者に分かる

10. Gradientで読みやすい色面を作る

装飾ではなく、現場で使える色面を設計する

第2巻 第10章 | レベル：初級 | 目安 20分 | 素材フォルダ：10_Gradient

ねらい

文字が載る部分を少し暗くした『読ませるための座布団』や、端に向かって透明にして映像になじむパネルを作る。派手さではなく、コントロールできる色面を目的から設計する。

STEP 1 目的を先に決める

Gradientは4色のリニアグラデーションで、透明度も含められる。作る前に目的を決める：上を少し明るく下を暗く／文字の後ろだけ暗くして読みやすい座布団／端に向かって透明にして映像になじませる、など。『きれいな背景』とだけ考えない。

確認：この色面の目的（なじませ／文字の座布団など）を言える。

STEP 2 情報が載る場所を先に置く

色より先に、文字や数字が載る位置を決める。仮の文字を先に置いてから色面を作ると、あとで読みにくくならない。位置が決まらないまま色を作ると、文字を載せたとき破綻しやすい。

確認：仮の文字が置かれ、載る位置が決まっている。

STEP 3 4色で役割を持たせる

- ① Material Editor で新規Materialを作り、Add Shader > Gradient を選ぶ（メニュー実在確認済み）
 - ② 4色（四隅）それぞれに役割を持たせて色を設定する：上2色を明るく・下2色を暗く（縦方向の変化だけ作る）／片側だけ暗くして文字置き場を作る、など
- 派手にするためではなく、明るさ・色味の変化に役割を持たせるのが目的。

確認：4色それぞれの役割を説明できる。

STEP 4 透明度で『なじませる』と『読ませる』を両立

- ① 各コーナーの色のアルファ（透明度）を調整する：外側の色だけアルファを下げて映像になじませる／文字が載る部分は不透明寄りを保つ
- ② 角1か所だけ透明度を変えて抜け感を作る、という使い方もできる

確認：端はなじみ、文字が載る部分は読める濃さになっている。

ヒント： 透明度は『なじませる』と『読ませる』のバランス。文字の下は読める濃さを確保する。

STEP 5 少しずつ変えて確認する

設定は少しずつ変える。派手に作るより、コントロールできる状態で作るのが大事。ローワーサードなら文字の下を暗く、スコア表示なら数字が読める濃さ、チームカラーは強すぎない範囲でなじませる。

確認：実際に文字を載せた状態で読める。

本番前チェックリスト

- ☐ 作る前に目的（読ませる/なじませる）を決めた
- ☐ 文字・数字の載る場所を先に置いてから色を作った
- ☐ 4色に役割を持たせた（変化の方向を絞った）
- ☐ 透明度で『なじませる』と『読ませる』を両立できている
- ☐ 文字の下が読める濃さになっている
- ☐ 派手さより、コントロールできる色面になっている

11. 現在時刻・カウントダウンを表示する

Date Timeで『変化する情報』として時間を扱う

第4巻 第5章 | レベル：中級 | 目安 30分 | 素材フォルダ：11_Clock_Countdown

ねらい

現在時刻の表示と、試合・イベント開始までのカウントダウンを作る。時間を『文字』ではなく『変化する情報』として扱い、目標時刻を作って現在時刻との差を出し、見やすい文字列に整える。

STEP 1 時間を『変化する情報』として捉える

実務の時間表示（開始時刻・開始日・カウントダウン・曜日・一定時刻での切替）は『時間に変化する』前提で作る。完成した1枚の絵ではなく、値が変わっても成立する仕組みを作る。

確認：作る表示が「変化する情報」であることを説明できる。

STEP 2 ClockかDate Timeブロックかを切り分ける

現在時刻や日付をそのまま出すだけなら Clock で足りる。「目標時刻との差（カウントダウン）」「日時を分解して曜日を出す」「指定した年月日から日時を作る」には Date Time系ブロックが要る。どちらもVisual Logic Editorの Function Blocks > Date Time カテゴリにある（実機のブロック一覧で確認済み）。

確認：Clockだけで足りるか、Date Time系ブロックが要るか判断できた。

STEP 3 現在時刻をそのまま出す（Clock）

- ① SceneのVisual Logic Editorを開く
- ② Function Blocks > Date Time から Clock をドラッグ
- ③ Clockの出力 → Format Date Time（表示用の文字列に整形）→ 表示用Textの Text に接続
- ④ 時・分・秒のうち必要な桁だけ出す書式にする

確認：現在時刻がTextに表示され、秒が進んでいる。

STEP 4 目標時刻を作る（Encode Date Time）

- ① Date Time の Encode Date Time をドラッグ
- ② 年・月・日・時・分・秒を入力（Math の Value ブロックで与える）して、試合開始時刻などの目標日時を組み立てる

確認：Encode Date Timeから目標日時が出力されている。

ヒント：月末±1か月のような境界（1/31の1か月後など）は期待通りになるか実機で確認。

STEP 5 差を計算して残り時間を出す (Time Delta)

- ① Date Time の Time Delta をドラッグ
- ② Clock (現在時刻) と Encode Date Time (目標時刻) を Time Delta に接続して差を計算
- ③ 出力の Days/Hours/Minutes/Seconds を使って残り時間を表示につなぐ

確認：残り時間 (目標－現在) が表示され、1秒ごとに減っていく。

STEP 6 表示用に整える (Format Date Time)

- ① Format Date Time で表示用の文字列に整える。曜日が要るなら Day of the Week で取り出す
- ② 桁数が変わると幅が動くので、Text Objectの幅と揃えを確認 (数字はFontのMonospace Numbersが安定)

確認：表示形式が意図とおりで、桁が変わっても幅が崩れない。

STEP 7 本番前に確認する

確認：現在時刻が正しく更新されるか／カウントダウンが0に近づくときの表示が破綻しないか／桁数が変わっても揃うか／タイムゾーンや基準時刻が意図通りか。値が変化し続ける前提で、しばらく置いて確認する。

本番前チェックリスト

- ☐ 時間を『変化する情報』として扱っている
- ☐ Clockで足りるか、Date Timeブロックが要るかを切り分けた
- ☐ Encode Date Timeで目標時刻を作れた
- ☐ Time Deltaで差を出し、残り時間を表示できた
- ☐ Format Date Timeで見やすく整え、桁揺れを確認した
- ☐ しばらく動かして更新・境界の破綻がないことを確認した

12. Render Viewで仮想モニターを作る

Sceneを素材化して演出に組み込む

第2巻 第4章 | レベル：中級 | 目安 30分 | 素材フォルダ：素材不要（機能だけで作成）

ねらい

別Sceneで作った情報表示（スコアや選手カード）を、演出Scene内のQuad＝仮想モニターに貼って表示する。素材Scene側を直すと演出側にも反映される「Sceneの部品化」を体験する。

STEP 1 素材になる情報Sceneを用意する

表示したい情報（例：スコア表示や選手カード）を、独立したSceneとして作る。名前はSRC_ScorePanelのように「部品」であることが分かる名前にする。文字が読める大きさで作り、背景は透過か単色にしておくと、貼ったときに扱いやすい。

確認：素材Sceneが単体で正しく表示される。演出側のSceneと同じ名前にしない（同名だとどちらが素材か分からなくなる。例：素材=SRC_ScorePanel／演出=RV_Monitor）。

STEP 2 演出Scene側に貼り先のQuadを置く

- ① Object Library の Primitives > Quad を演出Sceneにドラッグ
- ② Object Managerで Monitor_Main など役割が分かる名前に付ける
- ③ 少し角度を付けて置くと「画面の中のモニター」らしく見える

確認：Quadが意図した位置・角度で置かれている。

STEP 3 MaterialにRender View Shaderを追加する

Material Editorで新しいMaterialを作り、Add Shader > Render View を選ぶ。Scene / Camera Sourceで素材Sceneと使用するカメラを指定し、このMaterialをQuadに割り当てる。

確認：Quadの面に素材Sceneの絵が表示される。

ヒント：表示されないときは、素材Sceneの指定→カメラの指定→Materialの割り当て先、の順に確認する。素材側のCameraはOrtho
Cameraを選ぶと、パネルの文字が歪まず「読ませる画面」になる（第3巻のCamera選びの実践）。

STEP 4 Resolutionを使うサイズから決める

Render ViewのResolution（Width / Height）を、最終的に画面でどの大きさに見えるかから決める。実機での目安（1920×1080のパネルを大画面LEDに貼った場合）：100×100＝明確にぼけて読めない／480×270＝やや甘い／1920×1080＝鮮明。高すぎると描画負荷、低すぎると文字がぼける。

確認：一度100×100まで落としてぼけを体感し、文字が読める解像度に戻す。

STEP 5 Texture Coordinatesで収まりを調整する

貼った絵の位置・スケールをTexture Coordinatesで整え、パネルからはみ出しや余白を調整する。

確認：素材の端がQuadに正しく収まっている。

STEP 6 更新と参照関係を確認する

素材Scene側の文字や色を変更し、演出Scene側の仮想モニターに反映されることを確認する。

【実機確認済み】素材Sceneをリネームしても、Render View MaterialのScene参照は自動で追従する（名前の文字列ではなく内部参照。リネームで壊れない）。ただし削除は別なので、素材Sceneを消す前は使用先を必ず確認する。

最後に「どのSceneが、どのMaterialで、どのObjectに使われているか」を説明できる状態にしておく。

確認：素材側の変更が仮想モニターに反映される。リネームしても表示が維持される。

ヒント：Render Viewの多用は参照関係を複雑にする。「部品化する理由」を言えるものだけに使う。

本番前チェックリスト

- ☐ 素材SceneとMaterialに「部品」と分かる名前が付いているか
- ☐ Resolutionが使うサイズに合っているか（過剰・不足なし）
- ☐ 貼った状態で文字が読めるか
- ☐ 演出Scene全体の負荷に問題がないか
- ☐ 本番中に触る値と制作時に固定する値が分かれているか
- ☐ 素材Sceneを消したり改名したときの影響範囲を把握しているか

13. JSONで選手リストを回す

Table Presetsで階層データを表として扱う

第5巻 第3章 | レベル：中級 | 目安 40分 | 素材フォルダ：13_JSON_PlayerList

ねらい

選手リストのJSONをDataLinqで読み込み、選手名・背番号・ポジションをTextに表示する。さらにDataLinq Keyを使って「キーを1つ変えると行全体（名前・番号・ポジション）が連動して切り替わる」現場運用の形まで作る。素材は 13_JSON_PlayerList フォルダの players.json 一式を使用。

STEP 1 サンプルJSONを用意する

素材フォルダ 13_JSON_PlayerList の players.json を使う（UTF-8・BOMなし）。構造は team > players[] > name / name_en / number / position。6人目はわざと超長い名前にしてあり、幅崩れの確認に使える。

確認：エディタで開けて、構造（players配列の中に選手オブジェクト）を説明できる。

STEP 2 DataLinq ServerでJSON Sourceを追加する

XPression DataLinq Server（XPression本体とは別ウィンドウ）で Add DataLinq Source →一覧から「JSON DataLinq Source」（Provides access to JSON data files）を選びOK。

JSON Linq - Configuration で：

- File Path：players.json を指定
- Encoding：ASCII（既定）→ UTF-8 に必ず変更（ASCIIのままだと日本語が文字化けする）
- 更新方式：ローカルファイルなら Wait for file change

events（画面注記どおりネットワークドライブでは非推奨）、または Poll every 秒指定

確認：Sourceがエラーなく追加され、EncodingがUTF-8になっている。

ヒント： Enable Loggingを入れておくと、後でトラブルを追いやすい。

STEP 3 Table Presetsで「表（ファイル）」を登録する

Textに割り当てる前に先にやっておく（後の行連動でTable指定が正しくないと動かないため実質必須）。Table

Presetsは「JSON内の階層を指定する」ものではなく、複数のJSONファイルをName+File Pathで登録しておき、データ参照時のTable一覧で切り替えられるようにする仕組み（設定画面の説明にも「browsing for dataのTablesドロップダウンに出るTableの一覧」とある）。

手順：

① Settingsタブで Enable Table Presets にチェック → Table Presets タブを開く

② Add → Name列に名前、File Path列の「…」でJSONファイルを指定

③ 同梱サンプルなら次の4つを登録する（→の右はファイル名。File Pathには「…」で選んだ絶対パスが入る。相対パスは前提にしない）：

Players_Player → players.json

Players_Short → players_short.json

Players_MissingName → players_missing_name.json

Players_Empty → players_empty.json

「共通プレフィックス（Players_）+差分名」で揃えるのがコツ。あとでKeyによるTable切替（手順5の発展）が綺麗に組める。

④ OKで保存 → TextのBrowse（Select DataLinq Field）のTable一覧に4つが並び、選ぶだけで参照ファイルが切り替わる

確認：TableをPlayers_Emptyに切り替えると表示が空になり、Players_Playerに戻すと選手が復帰する。

ヒント： Name列だけ入れてFile Pathが空だと機能しない。File Pathは絶対パスになるため、別PCへ移すと崩れる。本番では全マシン共通の固定フォルダ（例 C:\XPression_Data\）にデータを置いて、そのパスで登録するのが定石（第6巻「納品で崩れない」参照）。

STEP 4 Textに割り当てる（Tableを選んでツリーから指定）

XPression側で選手名用のText Objectを選び、Object Inspectorの Data Source タブ → DataLinq を選択 → Set。

Set DataLinq Properties で Enabled にチェック → DataLinq一覧からこのJSONソースを選び → Browse。

Select DataLinq Field で、まず Table のドロップダウンで Players を選び、ツリーで players の中の name を選ぶ（上部Selectionに Column: players / Row: name / Table: Players と表示される） → OK。

背番号用Textには number、ポジション用には position を同様に割り当てる。

確認：「佐藤 大輝」「1」「GK」が文字化けせずに表示され、Table欄がPlayersになっている。

ヒント： 化けたときは手順2のEncoding（UTF-8か）を最初に疑う。Tableが違くと後のKey連動が動かないので、ここで必ず確認。

STEP 5 DataLinq Keyで行をまとめて切り替える（ここが肝・実機検証済み）

目的：キーを1つ変えるだけで、名前・背番号・ポジションが同じ選手の行にそろって切り替わる。

【確定事項（実機検証済み）】行指定は山かっこ+1始まりの行番号（2人目=players<2>）。値検索の書式（players<number=7>など）は通らない。players<%PlayerNo%>

が正解。Table欄が正しく（Players）設定されていないと動かない。

① Object ManagerでSceneを選択 → Object Inspectorの DataLinq Keys タブ → Add。Name に PlayerNo（Published にチェック、Value に初期値 2）。

② 各Text（name / number / position）の Set DataLinq Properties を開き、Column欄を players<%PlayerNo%> にする（%で囲む）。RowとTableはそのまま。

③ KeyのValueを変えて反映を確認する。反映タイミングは：プレビュー=即時／オンエア中のScene =一度Offlineにして再TAKEした時（テキスト差し替えと同じ鉄則）。

確認：Key値を2→3に変えると、プレビューが即時に高橋 悠人／11／FWへ連動して切り替わる。オンエア反映は再TAKEで。

ヒント： 範囲外の行番号（例 99）の挙動はWrap Indicesの設定で変わる（ON=有効な行に巻き戻る／OFF=欠損扱い）。【発展・実機検証済み】Table欄を Players_%TeamTable% の形にし、DataLinq Keysに TeamTable（値は Player / Short / Empty などの差分だけ）を追加すると、参照ファイルもKeyで切り替えられる。行のPlayerNoと合わせ、Key2つで「どの表の何行目」を完全制御できる。

STEP 6 Sequencerと外部（RossTalk）からKeyを変える

PublishedにしたKeyはSequencer（Take InspectorのTemplate Data）に公開され、オペレーターはそこに行番号を入れるだけで選手が切り替わる。

外部制御からはRossTalkの DATALINQKEY コマンドで同じことができる。

書式：DATALINQKEY [takeid]:[キー名]:[値]

実機検証済みの確実なシーケンス（Take ID 0002 を3行目の選手＝高橋 悠人に差し替えて送出）：

DATALINQKEY 0002:PlayerNo:3 ←①中身を高橋に差し替える

CUE 0002 ←②新しい値でTake Itemをキュー（積み込み）

（200ms程度待つ） ←③キュー完了を待つ

TAKE 2 ←④画面に出す（送出）

値を変えた直後にいきなりTAKEすると反映が間に合わないことがあるため、CUEと短いPauseを挟むのが確実。キー名は公開した名前と完全一致させる。

確認：Take InspectorでKey値を変えると表示が切り替わる。上のシーケンスを外部から送ると、新しい選手で確実に送出される。

ヒント： Keyの値はTake Item単位で保持される。同じSceneから複数のTake Itemを作れば、選手A用・B用として別々の行番号を持たせられる（外部システム連携の放送定番構成）。Stream Deck系ツール（Companion等）ならこの4手順を1ボタンに登録できる。

STEP 7 データ更新を確認する

players.json の値（名前や背番号）を書き換えて保存し、表示への反映を確認する。反映されないときは手順2の更新方式（Wait for file change events / Poll every）を確認する。

確認：保存後、想定したタイミングで表示が変わる。

STEP 8 壊れたデータで挙動を見る（Return Empty on Failureが鍵）

手順3でTable Presetsに登録した Players_Short（選手が減った）／Players_MissingName（nameが無い選手）／Players_Empty（空配列）にTableを切り替えて、表示を確認する。

【実機で確認された危険な挙動】各Textの Return Empty on Failure がOFFのままだと、行やデータが見つからないとき、Keyに入れた数字など意図しない値がそのまま画面に出る。

対策：DataLinqリンクした各Textの Set DataLinq Properties で Return Empty on Failure にチェック（ON）→ 欠損時は空欄になる。

Wrap Indicesとの関係も整理しておく：ON＝範囲外の行番号を有効な行に巻き戻す（何かしら表示される）／OFF＝範囲外は欠損扱い。「絶対に間違った選手を出したくない」ならWrap Indices OFF＋Return Empty ONの組み合わせが安全側。

確認：Return Empty on Failure をONにすると、欠損時に変な値ではなく空欄になる。

ヒント：空欄になったときにレイアウトが破綻しないか（座布団だけ残る等）も見ておく。原因調査には Enable Logging / Data Logger を使う。

本番前チェックリスト

- ☐ EncodingがUTF-8になっているか（既定ASCIIのままにしない）
- ☐ DataLinq Key名（PlayerNo等）が意味の分かる名前でPublishされているか
- ☐ Table Presets（Name＋File Path）の切替がTable一覧で機能するか
- ☐ 空データ・項目欠落時の表示を確認したか
- ☐ 更新方式（file change / Poll）を本番機で確認したか
- ☐ 項目名や階層の変更ルールを外部システム側と合意しているか

14. RossTalkで外部からTakeする

外部制御の入口をTCPで体験する

第5巻 第6章 | レベル：中級 | 目安 30分 | 素材フォルダ：14_RossTalk

ねらい

Smart GPI / RossTalkをTCPで受けられるようにし、外部からTAKE / SEQI / SEQOなどのコマンドでTake Itemを出し入れする。外部制御の基本動作と消し方の違いを体験する。

STEP 1 Hardware SetupでRossTalkを有効化する

メニューの Edit → Hardware Setup を開き、「GPI / Tally Boards」タブで Add → Brand一覧から Smart GPI / RossTalk を選ぶ（一覧にはPBus / Serial GPI (CTS/DSR) / TSL Tallyも並ぶ）。Smart GPI / RossTalk Setup で State を Enabled、Mode を TCP にし、Incoming Network Settings の TCP Port（例 7788）を控える。設定後、一覧の State が Active になればOK。

確認：State=Active・Mode=TCP・ポート番号をメモした。

ヒント：本番機ではWindowsファイアウォールの受信許可も確認する。Keyboard / GPI Mappingも同じEditメニューにある。

STEP 2 送出対象のTake Itemを用意する

- ① SequencerにテストSceneをドラッグしてTake Item化する
- ② Take ID列で番号を確認・設定（例 0002）。出し先のフレームバッファとレイヤーも確認
- ③
まず手動のTake（テンキー+）で正しく出ることを確かめておく（外部制御の前に手動で動くこと）

確認：手動のTake（テンキー+）で正しく出る状態になっている。

STEP 3 外部からTCP接続する（DashBoardが実戦的）

送信側はRoss公式の無料ツール DashBoard のCustomPanelが実戦的：ボタンを作り、TaskにRossTalkを選ぶ→Connection=Custom、Host=XPressionのIPアドレス（同一PCならlocalhost）、Port=手順1のポート番号。CommandはドロップダウンにTAKE / CUE / SEQI…が用意されており、Take IDなどを入れるだけ。

ターミナルソフト（PuTTYのRaw接続等）で直接コマンドを打つ方法もある（その場合はCR/LF終端に注意）。

確認：ボタン実行（または接続）がエラーなく通る。

ヒント： DashBoardならPause（待ち時間）もタスクとして挟めるので、「DATA LINQKEY→CUE→Pause 200ms→TAKE」のような複数コマンドを1ボタンに登録できる。素材フォルダ 14_RossTalk の検証済みサンプルパネルを開けば、Host/Portを直すだけで全ボタンを試せる。

STEP 4 TAKEで出す

「TAKE 1」または「TAKE 1:0:7」（TakeID:フレームバッファ:レイヤー）を送信し、オンエアされることを確認する。TAKEはSequencerのフォーカス（選択行）が動かないのが特徴。

確認：コマンド送信でSceneが出て、Sequencerの選択行は動かない。

STEP 5 SEQIとSEQOを試す

SEQI 1（テンプレ定義の出力先に出る。フォーカスはその項目へ移動）と、SEQO 1（そのTake IDを下げる）を送信して、TAKEとの違いを確認する。

確認：SEQIでは選択行が移動し、SEQOで消える。

STEP 6 片付け系コマンドを確認する

LAYEROFF 0000:7（そのレイヤーのSceneをOutトランジションで下げる）、CLFB 0000（フレームバッファをクリア）、CLRA（全フレームバッファをクリア）を試し、消え方の違いを見る。

確認：LAYEROFFはOutの動きで消え、CLFB / CLRAは即座にクリアされる。

ヒント：

CLRAは全部消える強いコマンド。本番での使いどころ（使わないという判断も含めて）を決めておく。

STEP 7 GPIコマンドで機能を呼ぶ（任意・物理機器は不要）

RossTalkの「GPI 1」はソフト的にGPI入力を叩くシミュレートコマンドで、物理的なGPI機器は不要。

① Edit → Keyboard / GPI Mapping

で、シミュレートGPI入力1に機能（例：Sequencerの選択を進める）を割り当てる

② DashBoardのRossTalkタスク（またはターミナル）から GPI 1 を送信

確認：GPIコマンドで、割り当てた機能が実行される。

本番前チェックリスト

- ☐ ポート番号とファイアウォールを本番機で確認したか
- ☐ コマンドのCR/LF終端を送信側が守っているか
- ☐ Take IDの付番ルール（範囲で意味を持たせる）が決まっているか
- ☐ TAKEとSEQIのフォーカス挙動の違いを運用者が説明できるか
- ☐ 本番中に接続が切れたことに気づける手順があるか
- ☐ 誤送信時のリカバリ（SEQO / LAYEROFF）を確認したか